SUST_Ragebait_Tree

Shahjalal University of Science and Technology

1 Data Structure **1**

1.1 Fenwick Tree 1
1.2 Fenwick Tree 2D Index Compression 1
1.3 Fenwick 2D 1
1.4 Lazy Propagation 1
1.5 Persistent Segment Tree . . 1
1.6 Persistent Lazy Propagation 2
1.7 Segment Tree 2D 2
1.8 Segment Tree 2D (Index compression) 2
1.9 Merse Sort Tree with Update 3
1.10 Treap 3
1.11 Persistent DSU 3
1.12 MOs Algorithm 4
1.13 MOs with updates 4
1.14 MOs on Trees 4
1.15 Dynamic Connectivity . . . 4
1.16 Link Cut Tree 4
1.17 Wavelet Tree 5
1.18 Sparse Table (RMQ) 5

1 Data Structure

1.1 Fenwick Tree

Description: Use for point update + range query or range update + point query. **1-indexed**.
Time: $O(\log N)$

```
struct FT {
  int n; vector<ll> s;
  FT(int n) : n(n + 1), s(n + 1) {}
  void update(int pos, ll dif) {
    for (; pos < n; pos += pos & -pos) s
     [pos] += dif;
  }
  ll query(int pos) {
    ll res = 0;
    for (; pos > 0; pos -= pos & -pos)
     res += s[pos];
    return res;
  }
  ll query(int l, int r) {
    return query(r) - query(l - 1);
  }
  void update(int l, int r, ll dif) {
    update(l, dif);
    if (r + 1 < n) update(r + 1, -dif);
  }
  int lower_bound(ll sum) {
    if (sum <= 0) return 0;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1)
      {
      if (pos + pw < n && s[pos+pw] <
     sum) {
        pos |= pw, sum -= s[pos];
      }
    }
    return pos + 1;
  }
};
```

1.2 Fenwick Tree 2D Index Compression

Description: Computes sums $a[i,j]$ for all $i \leq I, j \leq J$, and increases single elements $a[i,j]$. Requires that the elements to be updated are known in advance (call `fakeUpdate()` before `init()`).
Time: $O(\log^2 N)$. (Use persistent segment trees for $O(\log N)$)

```
struct FT2 {
  int n; vector<FT> ft;
  vector<vector<int>> ys;
  FT2(int limx) : n(limx + 1), ys(limx +
      1) {}
  void fakeUpdate(int x, int y) {
    for (; x < n; x += x & -x) {
      ys[x].push_back(y);
    }
  }
  void init() {
    ft.assign(ys.size(), FT(0));
    for (int i = 0; i < n; ++i) {
      sort(ys[i].begin(), ys[i].end());
      ys[i].erase(unique(ys[i].begin(),
     ys[i].end()), ys[i].end());
      ft[i] = FT(ys[i].size());
    }
  }
  int ind(int x, int y) {
    return upper_bound(ys[x].begin(), ys
     [x].end(), y) - ys[x].begin();
  }
  void update(int x, int y, ll dif) {
    for (; x < n; x += x & -x)
      ft[x].update(ind(x, y), dif);
  }
  ll query(int x, int y) {
    ll sum = 0;
    for (; x > 0; x -= x & -x) {
      sum += ft[x].query(ind(x, y));
    }
    return sum;
  }
  ll query(int x1, int y1, int x2, int
     y2) {
    return query(x2,y2) - query(x1-1, y2
    ) -
        query(x2, y1-1) + query(x1-1, y1
    -1);
  }
};
```

1.3 Fenwick 2D

```
struct FT2D {
  int n, m; vector<vector<ll>> s;
  FT2D(int n, int m) : n(n + 1), m(m +
     1), s(n + 1, vector<ll>(m + 1)) {}
  FT2D(int n) : FT2D(n, n) {}
  void update(int x, int y, ll dif) {
    for (int i = x; i < n; i += i & -i)
      for (int j = y; j < m; j += j & -j
    ) s[i][j] += dif;
  }
  ll query(int x, int y) {
    ll res = 0;
    for (int i = x; i > 0; i -= i & -i)
      for (int j = y; j > 0; j -= j & -j
    ) res += s[i][j];
    return res;
  }
  ll query(int x1, int y1, int x2, int
     y2) {
    return query(x2, y2) - query(x1 - 1,
     y2) - query(x2, y1 - 1) + query(x1
     - 1, y1 - 1);
  }
};
```

1.4 Lazy Propagation

```
struct ST {
  #define lc (n << 1)
  #define rc ((n << 1) | 1)
  vector<ll> a, t, lazy;
  ST(vector<ll>& a) : a(a) {
    lazy.resize(4 * a.size() + 1, 0);
    t.resize(4 * a.size() + 1, 0);
  }
  ST(int n) {
    a.resize(n);
    lazy.resize(4 * n, 0);
    t.resize(4 * n, 0);
  }
  inline long long combine(long long a,
     long long b) {
    return (a + b) % mod;
  }
  inline void pull(int n) {
    t[n] = (t[lc] + t[rc]) % mod;
  }
  // Updates n and lazy directly
  inline void apply(int n, int b, int e,
      long long v) {
    v %= mod;
    lazy[n] = (lazy[n] + v) % mod;
    t[n] = (t[n] + (ll)(e - b + 1) * v)
     % mod;
  }
  inline void push(int n, int b, int e)
     {
    if (lazy[n] == 0) return;
    if (b != e) {
      int mid = (b + e) >> 1;
      apply(lc, b, mid, lazy[n]);
      apply(rc, mid + 1, e, lazy[n]);
    }
    lazy[n] = 0;
  }
  void build(int n, int b, int e) {
    lazy[n] = 0;
    if (b == e) {
      t[n] = a[b] % mod;
      return;
    }
    int mid = (b + e) >> 1;
    build(lc, b, mid);
    build(rc, mid + 1, e);
    pull(n);
  }
  void upd(int n, int b, int e, int i,
     int j, long long v) {
    if (j < b || e < i) return;
    if (i <= b && e <= j) {
      apply(n, b, e, v);
      return;
    }
    push(n, b, e);
    int mid = (b + e) >> 1;
    upd(lc, b, mid, i, j, v);
    upd(rc, mid + 1, e, i, j, v);
    pull(n);
  }
  long long query(int n, int b, int e,
     int i, int j) {
    if (i > e || b > j) return 0;
    if (i <= b && e <= j) return t[n];
    push(n, b, e);
    int mid = (b + e) >> 1;
    return combine(query(lc, b, mid, i,
     j), query(rc, mid + 1, e, i, j));
  }
};
```

1.5 Persistent Segment Tree

```
struct PST {
#define lc t[cur].l
#define rc t[cur].r
  struct node {
    int l = 0, r = 0, val = 0;
  } t[40 * N];
  int T = 0;
  int combine(int a, int b) { return a +
      b; } // change
  void pull(int cur) { t[cur].val =
     combine(t[lc].val, t[rc].val); }
  int build(int b, int e) {
    int cur = ++T;
    if (b == e) return cur;
    int mid = b + e >> 1;
    lc = build(b, mid);
    rc = build(mid + 1, e);
    pull(cur);
    return cur;
  }
  // Build tree from initial vector
  int build(int b, int e, const vector<
     int>& a) {
    int cur = ++T;
    if (b == e) {
      t[cur].val = a[b];
      return cur;
```

```
    }
    int mid = b + e >> 1;
    lc = build(b, mid, a);
    rc = build(mid + 1, e, a);
    pull(cur);
    return cur;
  }
  int upd(int pre, int b, int e, int i,
     int v) {
    int cur = ++T;
    t[cur] = t[pre];
    if (b == e) {
      t[cur].val = v; // += for adding
      return cur;
    }
    int mid = b + e >> 1;
    if (i <= mid) lc = upd(t[pre].l, b,
     mid, i, v);
    else rc = upd(t[pre].r, mid + 1, e,
     i, v);
    pull(cur);
    return cur;
  }
  // Batch update: applies a sorted
     vector of updates {index, delta} in
      range [ql, qr)
  int upd_batch(int pre, int b, int e,
     const vector<pair<int, int>>& up,
     int ql, int qr) {
    if (ql > qr) return pre;
    int cur = ++T;
    t[cur] = t[pre];
    if (b == e) {
      for (int i = ql; i <= qr; ++i) t[
     cur].val += up[i].second;
      return cur;
    }
    int mid = b + e >> 1;
    int sp = upper_bound(up.begin() + ql
     , up.begin() + qr + 1, mid,
      [](int m, const pair<int, int>& p)
      { return m < p.first; }) - up.
     begin();
    lc = upd_batch(t[pre].l, b, mid, up,
      ql, sp - 1);
    rc = upd_batch(t[pre].r, mid + 1, e,
      up, sp, qr);
    pull(cur);
    return cur;
  }
  // Range query on a single version
  int query(int cur, int b, int e, int l
     , int r) {
    if (!cur || l > e || r < b) return
     0; // Identity value (0 for sum,
     INF for min)
    if (l <= b && e <= r) return t[cur].
     val;
    int mid = b + e >> 1;
    return combine(query(lc, b, mid, l,
     r), query(rc, mid + 1, e, l, r));
  }
  // Query k-th smallest element between
      versions pre and cur
  int query_kth(int pre, int cur, int b,
      int e, int k) {
    if (b == e) return b;
    int cnt = t[lc].val - t[t[pre].l].
     val;
    int mid = b + e >> 1;
    if (cnt >= k) return query_kth(t[pre
     ].l, lc, b, mid, k);
    else return query_kth(t[pre].r, rc,
     mid + 1, e, k - cnt);
  }
#undef lc
#undef rc
};
```

1.6 Persistent Lazy Propagation

```
struct PST {
#define lc t[cur].l
#define rc t[cur].r
  struct node {
    int l = 0, r = 0;
    ll val = 0, lazy = 0;
  } t[250 * N];
  int T = 0;
  inline ll combine(ll a, ll b) {
    return a + b;
  }
  inline void pull(int cur) {
    t[cur].val = combine(t[lc].val, t[rc
     ].val);
  }
  int build(int b, int e) {
    int cur = ++T;
    if(b == e) return cur;
    int mid = b + e >> 1;
    lc = build(b, mid);
    rc = build(mid + 1, e);
    pull(cur);
    return cur;
  }
  void apply(int cur, int b, int e, ll v
     ) {
    t[cur].lazy += v;
    t[cur].val += (ll)(e - b + 1) * v;
  }
  void push(int cur, int b, int e) {
    if(!t[cur].lazy) return;
    if(b != e) {
      int mid = b + e >> 1;
      int nlc = ++T; t[nlc] = t[lc];
      int nrc = ++T; t[nrc] = t[rc];
      lc = nlc; rc = nrc;
      apply(lc, b, mid, t[cur].lazy);
      apply(rc, mid + 1, e, t[cur].lazy)
     ;
    }
    t[cur].lazy = 0;
  }
  int upd(int pre, int b, int e, int i,
     int j, ll v) {
    int cur = ++T;
    t[cur] = t[pre];
    if(i <= b && e <= j) {
      apply(cur, b, e, v);
      return cur;
    }
    push(cur, b, e);
    int mid = b + e >> 1;
    if(i <= mid) lc = upd(lc, b, mid, i,
      j, v);
    if(j > mid) rc = upd(rc, mid + 1, e,
      i, j, v);
    pull(cur);
    return cur;
  }
  // Propagating query
  ll query(int cur, int b, int e, int i,
      int j) {
    if(!cur || b > j || e < i) return 0;
    if(i <= b && e <= j) return t[cur].
     val;
    push(cur, b, e);
    int mid = b + e >> 1;
    return combine(query(lc, b, mid, i,
     j), query(rc, mid + 1, e, i, j));
  }
  // Non-propagating query
  ll query(int cur, int b, int e, int i,
      int j, ll add = 0) {
    if(!cur || b > j || e < i) return 0;
    if(i <= b && e <= j) return t[cur].
     val + (ll)(e - b + 1) * add;
    int mid = (b + e) >> 1;
    add += t[cur].lazy;
    return combine(query(lc, b, mid, i,
     j, add), query(rc, mid + 1, e, i, j
     , add));
  }
#undef lc
#undef rc
};
```

1.7 Segment Tree 2D

```
// USAGE
ST2D sg(NX, NY); // dimensions 1..NX,
     1..NY
sg.upd   (1, 1, NX, x, y, v);
sg.query (1, 1, NX, x1, x2, y1, y2);
struct ST2D {
  #define lc (nd << 1)
  #define rc ((nd << 1) | 1)
  #define lcy (ndy << 1)
  #define rcy ((ndy << 1) | 1)
  int NX, NY;
  vector<vector<long long>> t;
  ST2D(int NX, int NY) : NX(NX), NY(NY),
      t(4 * NX, vector<long long>(4 * NY
     , 0)) {}
  inline long long combine(long long a,
     long long b) { return max(a, b); }
  inline void pull_y(int nd, int ndy) {
    t[nd][ndy] = combine(t[nd][lcy], t[
     nd][rcy]);
  }
  inline void pull_x(int nd, int ndy) {
    t[nd][ndy] = combine(t[lc][ndy], t[
     rc][ndy]);
  }
  void upd_y(int nd, int ndy, int by,
     int ey, int j, long long v, bool
     leaf_x) {
    if (by == ey) {
      if (leaf_x) t[nd][ndy] = v;
      else        t[nd][ndy] = combine(t
     [lc][ndy], t[rc][ndy]);
      return;
    }
    int mid = (by + ey) >> 1;
    if (j <= mid) upd_y(nd, lcy, by, mid
     , j, v, leaf_x);
    else          upd_y(nd, rcy, mid+1,
     ey, j, v, leaf_x);
    pull_y(nd, ndy);
  }
  long long query_y(int nd, int ndy, int
      by, int ey, int y1, int y2) {
    if (y1 > ey || by > y2) return 0;
    if (y1 <= by && ey <= y2) return t[
     nd][ndy];
    int mid = (by + ey) >> 1;
    return combine(query_y(nd, lcy, by,
     mid, y1, y2),
                   query_y(nd, rcy, mid
     +1, ey, y1, y2));
  }
  void upd(int nd, int bx, int ex, int i
     , int j, long long v) {
    if (bx == ex) {
      upd_y(nd, 1, 1, NY, j, v, true);
      return;
    }
    int mid = (bx + ex) >> 1;
    if (i <= mid) upd(lc, bx, mid, i, j,
      v);
    else          upd(rc, mid+1, ex, i,
     j, v);
    upd_y(nd, 1, 1, NY, j, v, false);
     // pull x into inner nodes
  }
  long long query(int nd, int bx, int ex
     , int x1, int x2, int y1, int y2) {
    if (x1 > ex || bx > x2) return 0;
    if (x1 <= bx && ex <= x2) return
     query_y(nd, 1, 1, NY, y1, y2);
    int mid = (bx + ex) >> 1;
    return combine(query(lc, bx, mid, x1
     , x2, y1, y2),
                   query(rc, mid+1, ex,
     x1, x2, y1, y2));
  }
};
```

1.8 Segment Tree 2D (Index compression)

```
// USAGE
ST2D seg(n);
// Phase 1: register all (x, y) points
     that will ever be updated
for (auto& [x, y, v] : updates)
    seg.collect(1, 1, n, x, y);
// Phase 2: compress and allocate inner
     trees
seg.build(1, 1, n);
// Updates and queries
seg.upd   (1, 1, n, x, y, v);
seg.query (1, 1, n, x1, x2, y1, y2);
////////////////////////////////////////
```

```
void compress(vector<int>& a) {
    sort(all(a));
    a.erase(unique(all(a)), a.end());
}
int ind(vi& a, int val) {
    return lower_bound(all(a), val) - a.
     begin();
}
struct ST2D {
  #define lc (nd << 1)
  #define rc ((nd << 1) | 1)
  int n;
  vector<vi> ys;       // compressed y-
     coords per x-node
  vector<vi> t;        // inner seg tree
      per x-node
  ST2D(int n) : n(n), ys(4 * n), t(4 * n
     ) {}
  void collect(int nd, int b, int e, int
      x, int y) {
    ys[nd].pb(y);
    if (b == e) return;
    int mid = (b + e) >> 1;
    if (x <= mid) collect(lc, b, mid, x,
      y);
    else          collect(rc, mid + 1, e
     , x, y);
  }
  void build(int nd, int b, int e) {
    compress(ys[nd]);
    t[nd].assign(ys[nd].sz() * 2, 0);
    if (b == e) return;
    int mid = (b + e) >> 1;
    build(lc, b, mid);
    build(rc, mid + 1, e);
  }
  inline long long combine(long long a,
     long long b) { return max(a, b); }
  void upd_y(int nd, int y, long long v)
      {
    int m = ys[nd].sz();
    int i = ind(ys[nd], y) + m;
    t[nd][i] = combine(t[nd][i], v);
    for (i >>= 1; i >= 1; i >>= 1)
      t[nd][i] = combine(t[nd][i*2], t[
     nd][i*2+1]);
  }
  long long query_y(int nd, int y1, int
     y2) {
    if (y1 > y2) return 0;
    int m = ys[nd].sz();
    int l = ind(ys[nd], y1) + m;
    int r = (int)(upper_bound(all(ys[nd
     ]), y2) - ys[nd].begin() - 1) + m;
    ll mx = 0;
    while (l <= r) {
        if (l%2 == 1) mx = combine(mx, t
     [nd][l++]);
        if (r%2 == 0) mx = combine(mx, t
     [nd][r--]);
        l >>= 1; r >>= 1;
    }
    return mx;
  }
  void upd(int nd, int b, int e, int x,
     int y, long long v) {
    upd_y(nd, y, v);
    if (b == e) return;
    int mid = (b + e) >> 1;
    if (x <= mid) upd(lc, b, mid, x, y,
     v);
    else          upd(rc, mid + 1, e, x,
      y, v);
  }
  long long query(int nd, int b, int e,
     int x1, int x2, int y1, int y2) {
    if (x1 > x2 || y1 > y2) return 0;
    // empty range (e.g. a[i].f == 1)
    if (x1 > e  || b > x2)  return 0;
    if (x1 <= b && e <= x2)  return
     query_y(nd, y1, y2);
    int mid = (b + e) >> 1;
    return combine(query(lc, b, mid, x1,
      x2, y1, y2),
                   query(rc, mid+1, e,
     x1, x2, y1, y2));
}
};
```

1.9 Merse Sort Tree with Update

```
// 0 based indexing
template <typename T>
struct MST {
  int n;
  vector<ordered_set<pair<T, int>>> tree
     ;
  vector<T> a;
  MST(vector<T>& v) : a(v) {
    n = v.size();
    tree.resize(4 * n);
    build(0, 0, n - 1, a);
  }
  void merge(int u, int l, int r) {
    tree[u] = tree[l];
    for (auto x : tree[r]) tree[u].
     insert(x);
  }
  void build(int u, int l, int r, vector
     <T>& a) {
    if (l == r) {
      tree[u].insert({a[l], l});
      return;
    }
    int mid = (l + r) / 2;
    build(2 * u + 1, l, mid, a);
    build(2 * u + 2, mid + 1, r, a);
    merge(u, 2 * u + 1, 2 * u + 2);
  }
  // Returns number of elements smaller
     than x in range l, r
  int query(int l, int r, T x) { return
     get(0, 0, n - 1, l, r, x); }
  void update(int i, T x) {
    _update(0, 0, n - 1, i, x);
    a[i] = x;
  }
  int get(int u, int tl, int tr, int l,
     int r, T x) {
    if (tl >= l && tr <= r) return tree[
     u].order_of_key({x, 0});
    if (tl > r || tr < l) return 0;
    int mid = (tl + tr) / 2;
    return get(2 * u + 1, tl, mid, l, r,
      x) +
           get(2 * u + 2, mid + 1, tr, l
     , r, x);
  }
  void _update(int u, int l, int r, int
     i, T x) {
    tree[u].erase({a[i], i});
    tree[u].insert({x, i});
    if (l == r) return;
    int mid = (l + r) / 2;
    if (i <= mid)
      _update(2 * u + 1, l, mid, i, x);
    else
      _update(2 * u + 2, mid + 1, r, i,
     x);
  }
};
```

1.10 Treap

```
struct ImplicitTreap {
  struct Node { int l, r, sz, sum, val,
     priority, lazy; };
  vector<Node> tree;
  mt19937 rnd;
  ImplicitTreap(int max_nodes = 0) {
    rnd.seed(chrono::steady_clock::now()
     .time_since_epoch().count());
    if (max_nodes > 0) tree.reserve(
     max_nodes);
  }
  int getSize(int t) { return t == -1 ?
     0 : tree[t].sz; }
  int getSum(int t) { return t == -1 ? 0
      : tree[t].sum; }
  void applyLazy(int t, int add) {
    if (t == -1) return;
    tree[t].val += add; tree[t].sum +=
     tree[t].sz * add; tree[t].lazy +=
     add;
  }
  void pull(int t) {
    if (t == -1) return;
    tree[t].sz = 1 + getSize(tree[t].l)
     + getSize(tree[t].r);
    tree[t].sum = tree[t].val + getSum(
     tree[t].l) + getSum(tree[t].r);
  }
  void push(int t) {
    if (t == -1 || tree[t].lazy == 0)
     return;
    applyLazy(tree[t].l, tree[t].lazy);
     applyLazy(tree[t].r, tree[t].lazy);
    tree[t].lazy = 0;
  }
  int newNode(int val) {
    tree.push_back({-1, -1, 1, val, val,
      (int)rnd(), 0});
    return tree.size() - 1;
  }
  pair<int, int> split(int t, int k) {
    if (t == -1) return {-1, -1};
    push(t);
    if (getSize(tree[t].l) >= k) {
      auto res = split(tree[t].l, k);
      tree[t].l = res.second; pull(t);
      return {res.first, t};
    } else {
      auto res = split(tree[t].r, k -
     getSize(tree[t].l) - 1);
      tree[t].r = res.first; pull(t);
      return {t, res.second};
    }
  }
  int merge(int a, int b) {
    if (a == -1) return b;
    if (b == -1) return a;
    if (tree[a].priority > tree[b].
     priority) {
      push(a); tree[a].r = merge(tree[a
     ].r, b); pull(a);
      return a;
    } else {
      push(b); tree[b].l = merge(a, tree
     [b].l); pull(b);
      return b;
    }
  }
  int insert(int root, int pos, int val)
      {
    int nn = newNode(val);
    auto [L, R] = split(root, pos);
    return merge(merge(L, nn), R);
  }
  int erase(int root, int pos) {
    auto [L, R] = split(root, pos);
    auto [p, q] = split(R, 1);
    return merge(L, q);
  }
  void show(int t) {
    if (t == -1) return;
    push(t); show(tree[t].l);
    cout << tree[t].val << " "; show(
     tree[t].r);
  }
};
```

1.11 Persistent DSU

```
struct DSU {
  vector<vector<pair<int, int>>> par;
  int time = 0; //initial time
  DSU(int n) : par(n + 1, {{-1, 0}}) {}
  bool merge(int u, int v) {
    ++time;
    if ((u = root(u, time)) == (v = root
     (v, time))) return 0;
    if (par[u].back().first > par[v].
     back().first) swap(u, v);
    par[u].push_back({par[u].back().
     first + par[v].back().first, time})
     ;
    par[v].push_back({u, time}); //par[v
     ] = u
    return 1;
  }
  bool same(int u, int v, int t) {
    return root(u, t) == root(v, t);
  }
  int root(int u, int t) { //root of u
```

```
    at time t
    if (par[u].back().first >= 0 && par[
    u].back().second <= t) return root(
    par[u].back().first, t);
    return u;
  }
  int size(int u, int t) { //size of the
     component of u at time t
    u = root(u, t);
    int l = 0, r = (int) par[u].size() -
     1, ans = 0;
    while (l <= r) {
      int mid = l + r >> 1;
      if (par[u][mid].second <= t) ans =
     mid, l = mid + 1;
      else r = mid - 1;
    }
    return -par[u][ans].first;
  }
};
```

1.12 MOs Algorithm

```
void add(int ind, int end, int L, int R)
     { /* ... */ }
void del(int ind, int end, int L, int R)
     { /* ... */ }
int calc(int L, int R) { /* ... */ }
vector<int> mo(vector<pair<int, int>>& Q
    ) {
  int L = 1, R = 0, blk = 350;
  vector<int> s(Q.size()), res(Q.size())
    ;
  iota(s.begin(), s.end(), 0);
  sort(s.begin(), s.end(), [&](int i,
    int j) {
    int bi = Q[i].first / blk;
    int bj = Q[j].first / blk;
    if (bi != bj) return bi < bj;
    if (bi & 1) return Q[i].second < Q[j
    ].second;
    return Q[i].second > Q[j].second;
  });
  for (int qi : s) {
    pair<int, int> q = Q[qi];
    int ql = q.first;
    int qr = q.second;
    while (L > ql) { --L; add(L, 0, L, R
    ); }
    while (R < qr) { ++R; add(R, 1, L, R
    ); }
    while (L < ql) { del(L, 0, L, R); L
    ++; }
    while (R > qr) { del(R, 1, L, R); R
    --; }
    res[qi] = calc(L, R);
  }
  return res;
}
```

1.13 MOs with updates

```
struct mos_update {
  ll a[100005];
  ll ans;
  mos_update() { /* ... */ }
  void add(int ind, int end, int L, int
    R) { /* ... */ }
  void del(int ind, int end, int L, int
    R) { /* ... */ }
  void apply(int pos, int val, int L,
    int R) {
    if (L <= pos && pos <= R) {
      del(pos, -1, L, R); a[pos] = val;
    add(pos, -1, L, R);
    } else {
      a[pos] = val;
    }
  }
  vector<ll> mo(vector<vector<int>>& Q,
    vector<vector<int>>& U, int n) {
    int L = 1, R = 0, T = 0;
    ans = 0;
    int blk = max(1, (int)pow(Q.size(),
    0.666));
    vector<int> s(Q.size());
    vector<ll> res(Q.size());
    iota(s.begin(), s.end(), 0);
    sort(s.begin(), s.end(), [&](int i,
    int j) {
      int li = Q[i][0] / blk, lj = Q[j
    ][0] / blk;
      if (li != lj) return li < lj;
      int ri = Q[i][1] / blk, rj = Q[j
    ][1] / blk;
      if (ri != rj) return (li & 1) ? ri
     < rj : ri > rj;
      return (ri & 1) ? Q[i][2] < Q[j
    ][2] : Q[i][2] > Q[j][2];
    });
    for (int qi : s) {
      int ql = Q[qi][0], qr = Q[qi][1],
    qt = Q[qi][2];
      while (T < qt) { apply(U[T][0], U[
    T][2], L, R); T++; }
      while (T > qt) { --T; apply(U[T
    ][0], U[T][1], L, R); }
      while (L > ql) { --L; add(L, 0, L,
     R); }
      while (R < qr) { ++R; add(R, 1, L,
     R); }
      while (L < ql) { del(L, 0, L, R);
    L++; }
      while (R > qr) { del(R, 1, L, R);
    R--; }
      res[qi] = ans;
    }
    return res;
  }
};
```

1.14 MOs on Trees

```
// to be added
```

1.15 Dynamic Connectivity

```
struct ST {
#define lc (n << 1)
#define rc ((n << 1) | 1)
  vector<query> t[maxN*4];
  ST(int n) {}
  void upd(int n, int b, int e, int ql,
     int qr, query& v) {
    if(ql <= b && e <= qr) {
        t[n].push_back(v);
        return;
    }
    int mid = (b + e) >> 1;
    if (ql <= mid) upd(lc, b, mid, ql,
    qr, v);
    if(qr > mid) upd(rc, mid + 1, e, ql,
     qr, v);
  }
  // can be generalized to any rollback
     structure
  void dfs(int n, int b, int e, int root
    , int eg) {
    int sz = t[n].size(), init = pst.T,
    croot = root;
    for(auto u : t[n]) croot = pst.upd(
    croot, 1, eg, u.a, u.b, u.h);
    if(b == e) {  // query ans
        if(!(ask[b][0] == 0 && ask[b][1]
     == 0))
            ans[b] = pst.query(croot, 1,
     eg, ask[b][0], ask[b][1]);
    } else {
        int mid = (b+e) >> 1;
        dfs(lc, b, mid, croot, eg); dfs(
    rc, mid+1, e, croot, eg);
    }
    pst.T = init; // rollback
  }
  #undef lc
  #undef rc
};
```

1.16 Link Cut Tree

```
#include <bits/stdc++.h>
using namespace std;
const long long INF = 1e18;
struct node {
  int p = 0, c[2] = {0, 0}, sz = 0, ssz
     = 0, vsz = 0, mn_id = 0;
  bool flip = false;
  long long val = INF, sum = 0, lazy =
     0, subsum = 0, vsum = 0, mn = INF;
  node() {}
  node(int id, long long x) : val(x),
     sum(x), subsum(x), mn(x), mn_id(id)
     , sz(1), ssz(1) {}
};
struct LCT {
  vector<node> t;
  LCT(int n, int m = 0) : t(n + m + 1) {
    for (int i = 1; i <= n + m; i++) t[i
    ] = node(i, INF);
    t[0] = node();
    t[0].sz = t[0].ssz = t[0].vsz = t
    [0].sum = t[0].subsum = t[0].vsum =
     0;
  }
  bool is_root(int x) { return t[t[x].p
    ].c[0] != x && t[t[x].p].c[1] != x;
     }
  int dir(int x) { return t[t[x].p].c[1]
     == x; }
  void flip(int x) {
    if (x) swap(t[x].c[0], t[x].c[1]), t
    [x].flip ^= 1;
  }
  void add(int x, long long v) {
    if (!x) return;
    t[x].val += v; t[x].mn += v; t[x].
    lazy += v;
    t[x].sum += v * t[x].sz; t[x].subsum
     += v * t[x].ssz; t[x].vsum += v *
    t[x].vsz;
  }
  void push(int x) {
    if (!x) return;
    if (t[x].flip) flip(t[x].c[0]), flip
    (t[x].c[1]), t[x].flip = false;
    if (t[x].lazy) add(t[x].c[0], t[x].
    lazy), add(t[x].c[1], t[x].lazy), t
    [x].lazy = 0;
  }
  void pull(int x) {
    if (!x) return;
    int l = t[x].c[0], r = t[x].c[1];
    t[x].sz = t[l].sz + t[r].sz + 1;
    t[x].ssz = t[l].ssz + t[r].ssz + t[x
    ].vsz + 1;
    t[x].sum = t[l].sum + t[r].sum + t[x
    ].val;
    t[x].subsum = t[l].subsum + t[r].
    subsum + t[x].vsum + t[x].val;
    t[x].mn = t[x].val; t[x].mn_id = x;
    if (t[l].mn < t[x].mn) t[x].mn = t[l
    ].mn, t[x].mn_id = t[l].mn_id;
    if (t[r].mn < t[x].mn) t[x].mn = t[r
    ].mn, t[x].mn_id = t[r].mn_id;
  }
  void push_all(int x) {
    if (!is_root(x)) push_all(t[x].p);
    push(x);
  }
  void rotate(int x) {
    int y = t[x].p, z = t[y].p, dx = dir
    (x), dy = dir(y), w = t[x].c[!dx];
    if (!is_root(y)) t[z].c[dy] = x;
    t[x].p = z; t[x].c[!dx] = y; t[y].p
    = x; t[y].c[dx] = w;
    if (w) t[w].p = y;
    pull(y); pull(x);
  }
  void splay(int x) {
    push_all(x);
    while (!is_root(x)) {
      int y = t[x].p;
      if (!is_root(y)) rotate(dir(x) ==
    dir(y) ? y : x);
      rotate(x);
    }
  }
  int access(int u) {
    int last = 0;
    for (int v = 0; u; v = u, u = t[u].p
    ) {
      splay(u);
      int r = t[u].c[1];
      t[u].vsz += t[r].ssz - t[v].ssz;
      t[u].vsum += t[r].subsum - t[v].
```

```
    subsum;
      t[u].c[1] = v; pull(u); last = u;
    }
    return last;
  }
  void make_root(int u) { access(u);
     splay(u); flip(u); }
  int find_root(int u) {
    access(u); splay(u); push(u);
    while (t[u].c[0]) u = t[u].c[0],
     push(u);
    splay(u);
    return u;
  }
  bool connected(int u, int v) { return
     find_root(u) == find_root(v); }
  void link(int u, int v) {
    make_root(u);
    if (find_root(v) == u) return;
    access(v); splay(v);
    t[u].p = v; t[v].vsz += t[u].ssz; t[
     v].vsum += t[u].subsum; pull(v);
  }
  void cut(int u, int v) {
    make_root(u); access(v); splay(v);
    if (t[v].c[0] == u && !t[u].c[1]) t[
     v].c[0] = t[u].p = 0, pull(v);
  }
  // --- Operations ---
  void link_edge(int e, int u, int v,
     long long w) { set_val(e, w); link(
     u, e); link(e, v); }
  void cut_edge(int e, int u, int v) {
     cut(u, e); cut(e, v); }
  void set_val(int u, long long x) {
     access(u); splay(u); t[u].val = x;
     pull(u); }
  void add_val(int u, long long x) {
     access(u); splay(u); t[u].val += x;
      pull(u); }
  void add_path(int u, int v, long long
     x) { make_root(u); access(v); splay
     (v); add(v, x); }
  pair<long long, int> query_min(int u,
     int v) { make_root(u); access(v);
     splay(v); return {t[v].mn, t[v].
     mn_id}; }
  long long query_path_sum(int u, int v)
      { make_root(u); access(v); splay(v
     ); return t[v].sum; }
  long long query_subtree_sum(int u, int
      root = 1) { make_root(root);
     access(u); splay(u); return t[u].
     val + t[u].vsum; }
};
```

1.17 Wavelet Tree

```
struct wavelet_tree {
  int lo, hi;
  wavelet_tree *l = nullptr, *r =
     nullptr;
  vector<int> b, c;
  wavelet_tree() : lo(0), hi(0) {}
  void init(int *from, int *to, int x,
     int y) {
    lo = x; hi = y;
    if (from >= to) return;
    int mid = lo + (hi - lo) / 2;
    auto f = [mid](int val) { return val
      <= mid; };
    b.reserve(to - from + 1); c.reserve(
     to - from + 1);
    b.push_back(0); c.push_back(0);
    for (auto it = from; it != to; ++it)
      {
      b.push_back(b.back() + f(*it));
      c.push_back(c.back() + *it);
    }
    if (lo == hi) return;
    auto pivot = stable_partition(from,
     to, f);
    (l = new wavelet_tree())->init(from,
      pivot, lo, mid);
    (r = new wavelet_tree())->init(pivot
     , to, mid + 1, hi);
  }
  int kth(int l, int r, int k) {
    if (l > r) return 0;
    if (lo == hi) return lo;
    int inLeft = b[r] - b[l - 1], lb = b
     [l - 1], rb = b[r];
    if (k <= inLeft) return this->l->kth
     (lb + 1, rb, k);
    return this->r->kth(l - lb, r - rb,
     k - inLeft);
  }
  int LTE(int l, int r, int k) {
    if (l > r || k < lo) return 0;
    if (hi <= k) return r - l + 1;
    int lb = b[l - 1], rb = b[r];
    return l->LTE(lb + 1, rb, k) + r->
     LTE(l - lb, r - rb, k);
  }
  int count(int l, int r, int k) {
    if (l > r || k < lo || k > hi)
     return 0;
    if (lo == hi) return r - l + 1;
    int lb = b[l - 1], rb = b[r], mid =
     lo + (hi - lo) / 2;
    if (k <= mid) return l->count(lb +
     1, rb, k);
    return r->count(l - lb, r - rb, k);
  }
  int sum(int l, int r, int k) {
    if (l > r || k < lo) return 0;
    if (hi <= k) return c[r] - c[l - 1];
    int lb = b[l - 1], rb = b[r];
    return l->sum(lb + 1, rb, k) + r->
     sum(l - lb, r - rb, k);
  }
  ~wavelet_tree() { delete l; delete r;
     }
};
```

1.18 Sparse Table (RMQ)

```
template<typename T>
struct SparseTable {
  const int N = 1e5 + 9;
  vector<vector<T>> t;
  vector<T> a;
  T combine(T x, T y) { return min(x, y)
     ; }
  void build(int n) {
    t.assign(n + 1, vector<T>(18));
    for(int i = 1; i <= n; ++i) t[i][0]
     = a[i];
    for(int k = 1; k < 18; ++k)
      for(int i = 1; i + (1 << k) - 1 <=
     n; ++i)
        t[i][k] = combine(t[i][k-1], t[i
     + (1 << (k-1))][k-1]);
  }
  T query(int l, int r) {
    int k = 31 - __builtin_clz(r - l +
     1);
    return combine(t[l][k], t[r - (1 <<
     k) + 1][k]);
  }
};
```